

BUILDING SECURE & SCALABLE DATATABLES APIS IN COLDFUSION

ColdFusion + DataTables + Secure Server-Side APIs

Who Am I?

Larry C. Lyons

larryclyons@gmail.com

- I've spent more than 20 years building secure, scalable, and maintainable ColdFusion applications that perform reliably in enterprise environments.
- Much of my work has focused on designing, maintaining, and modernizing ColdFusion applications using server-side architectures and API-driven integrations.

THE PROBLEM

- Client-side DataTables fails at scale
- Large datasets overwhelm browser memory
- Sorting and filtering become increasingly inefficient
- Server-side processing solves scalability problems

WHAT IS DATATABLES?

- DataTables (<http://datatable.net/>) is a Javascript HTML table enhancing library.
- It is a highly flexible tool, built upon the foundations of progressive enhancement.
- It provides built-in searching, sorting, filtering, pagination, and responsive layouts.
- Supports both client-side and server-side processing models.
- Widely used in enterprise applications for displaying large datasets.
- Integrates cleanly with JSON APIs and Bootstrap interfaces.

ARCHITECTURE

- Browser → DataTables
- ColdFusion API Layer
- Parameterized SQL Queries → Database
- JSON Response Back to DataTables
- Datatables populates the table.

DATATABLES INITIALIZATION

- Enable server-side processing
- Configure ajax endpoint
- Define responsive columns

```
let table = $('#employees').DataTable({
  autoWidth: true,
  processing: true,
  serverSide: true,
  rowId: 'employee_id',
  ajax: {
    url: 'com/Employees.cfc?method=getEmployeesJSON',
    type: 'POST',
    contentType: 'application/json; charset=utf-8',
    dataType: 'json',
    data: function(data) {
      return JSON.stringify(data);
    }
  },
});
```

DATATABLES INITIALIZATION

- Define responsive columns
- Column Rendering

```
{  
  title: 'Salary',  
  data: 'salary',  
  type: 'num-fmt',  
  searchable: true,  
  orderable: true,  
  render: function (data, type, row) {  
    let formattedAmount = new Intl.NumberFormat('en-US', {  
      style: 'currency',  
      currency: 'USD'  
    }).format(data);  
    return formattedAmount;  
  }  
},
```

COLDFUSION JVM ARGUMENT MATCHING

- Recent CF updates enforce strict remote argument matching.
- DataTables sends many dynamic request fields.
- Remote CFC methods must explicitly declare arguments.
- Undefined arguments now throw IllegalArgumentException errors.
- Impacts CF2025 U2, CF2023 U14, and CF2021 U20.

JVM Override

```
-Dcoldfusion.runtime.remotemethod.matchArguments=false
```

CommandBox server.json

```
"jvm":{  
  "args":[  
    "-Dcoldfusion.runtime.remotemethod.matchArguments=false"  
  ]  
}
```


THE COLDFUSION ENDPOINT & getEmployees() FUNCTION

- Single API endpoint handles requests
- Validate all incoming data
- Normalize pagination values
- Return JSON payloads

```
remote array function getEmployeesJSON(){

    // DataTables posts a JSON body so ColdFusion's strict
    // remote-argument matching does not reject the call
    // before this method can run.

    var dt = {
        "draw"      : 1,
        "start"     : 0,
        "length"    : 10
    };
    var http = getHttpRequestData();
    var rawBody = trim(http.content);
    dt = deserializeJSON(rawBody);

    // Additional validation logic
```

PAGING AND PAGE-SIZE SAFETY

- Uses SQL LIMIT and OFFSET
- Caps maximum page size
- Prevents excessive queries
- Ensures predictable API performance

```
// DataTables uses start=0;
var pageStart = max(0, int(val(dt.start ?: 0)));
var pageLength = int(val(dt.length ?: 10));

// length = -1 means "all"; we still cap
// it for safety.

if (pageLength EQ -1) {
    pageLength = variables.maxPageSize;
} else {
    pageLength = max(0, min(pageLength,
                           variables.maxPageSize));
}
```

SECURE ORDER BY WHITELIST

- Never trust browser column names
- Whitelist sortable columns
- Normalize sort direction
- Prevent ORDER BY injection

```
var columns = [  
    'employee_id',  
    'name',  
    'position'  
];  
  
var sortCol =  
    columns[val(arguments.order[1].column)];  
  
var sortDir =  
    arguments.order[1].dir == 'desc'  
    ? 'DESC'  
    : 'ASC';
```

GLOBAL SEARCH IMPLEMENTATION

- Search across indexed columns
- Use parameterized LIKE filters
- Avoid functions that break indexes
- Support DataTables global search

```
var whereClause = "WHERE 1 = 1";
var params      = {};
var hasSearch   = false;
if (structKeyExists(dt, "search[value]") &&
    len(trim(dt["search[value]"]))) {
    hasSearch = true;
    var searchTerms = trim(dt["search[value]"]);
    var searchUpper = uCase(searchTerms);
    params.sLike = { value = "%#searchUpper#%",
                     cfsqltype = "cf_sql_varchar" };
    params.dLike = { value = "%#searchTerms#%",
                     cfsqltype = "cf_sql_varchar" };
    var isNum = isNumeric(searchTerms);
    if (isNum) {
        params.nLike =
            { value = "%#trim(searchTerms)#%",
              cfsqltype = "cf_sql_varchar" };
    }
}
```

PAGINATION STRATEGY & THE SQL SCRIPT

- Use MySQL LIMIT / OFFSET
- Prevent massive page requests
- Always enforce maximum page size
- Database handles paging efficiently

```
var sql = "  
    SELECT id AS employee_id,  
           name,  
           position,  
           salary,  
           office,  
           extn,  
           DATE_FORMAT(start_date, '%m/%d/%Y')  
           AS start_date  
    FROM employees  
       #whereClause#  
   ORDER BY #orderColumn# #orderDir#  
   LIMIT :pageLength OFFSET :pageStart  
";
```

SECURE QUERYEXECUTE()

- Always use parameterized queries
- Typed bindings improve security
- Avoid string concatenation
- Supports efficient execution plans

```
var dataOptions = {  
    datasource      : variables.dsn,  
    returnType     : "array",  
    cachedWithin  : (hasSearch ?  
                      variables.cachePageSearch :  
                      variables.cachePageNoSearch)  
};  
  
// execute SQL – try catch removed.  
var employeeData =  
    QueryExecute(sql, params, dataOptions);
```

JSON RESPONSE FORMAT

- Must exactly match DataTables protocol
- Include draw and record counts
- Return array-based payloads
- Keep response structure consistent

```
Var results = {  
    "draw"           = int(val(dt.draw)),  
    "recordsTotal"   = int(totalRecords),  
    "recordsFiltered" = int(filteredRecords),  
    "data"           = employees  
};  
  
return serializeJSON(results);
```

COMMON MISTAKES

- String-concatenated SQL
- Unlimited page sizes
- Returning entire datasets
- Blind trust of request variables

Conclusions

- Validate every request
- Whitelist SQL structure
- Parameterize all user values
- Cap page sizes
- Cache strategically
- Treat the CFC as an API boundary

Key Takeaways

- Server-side DataTables is an API design problem, not just a JS Plugin
- Don't trust browser based requests.
- Order BY requires whitelisting because the identifiers. can't be parameterized.
- Parameterize all named values to avoid SQL injection.
- Paging limits are both security and performance controls.
- Cache strategically. Tune the caching differently to total counts, filtered counts, searched and unsearched pages.
- `ReturnType="array"` in the query options produces a JSON-friendly shape that is readily converted to JSON and consumed by DataTables directly.

Thank you for attending

Optional Q & A Prompts.

- SQL Server vs MySQL
- Column-specific searching
- Role-based filtering
- Indexing strategies
- JSON serialization approaches